

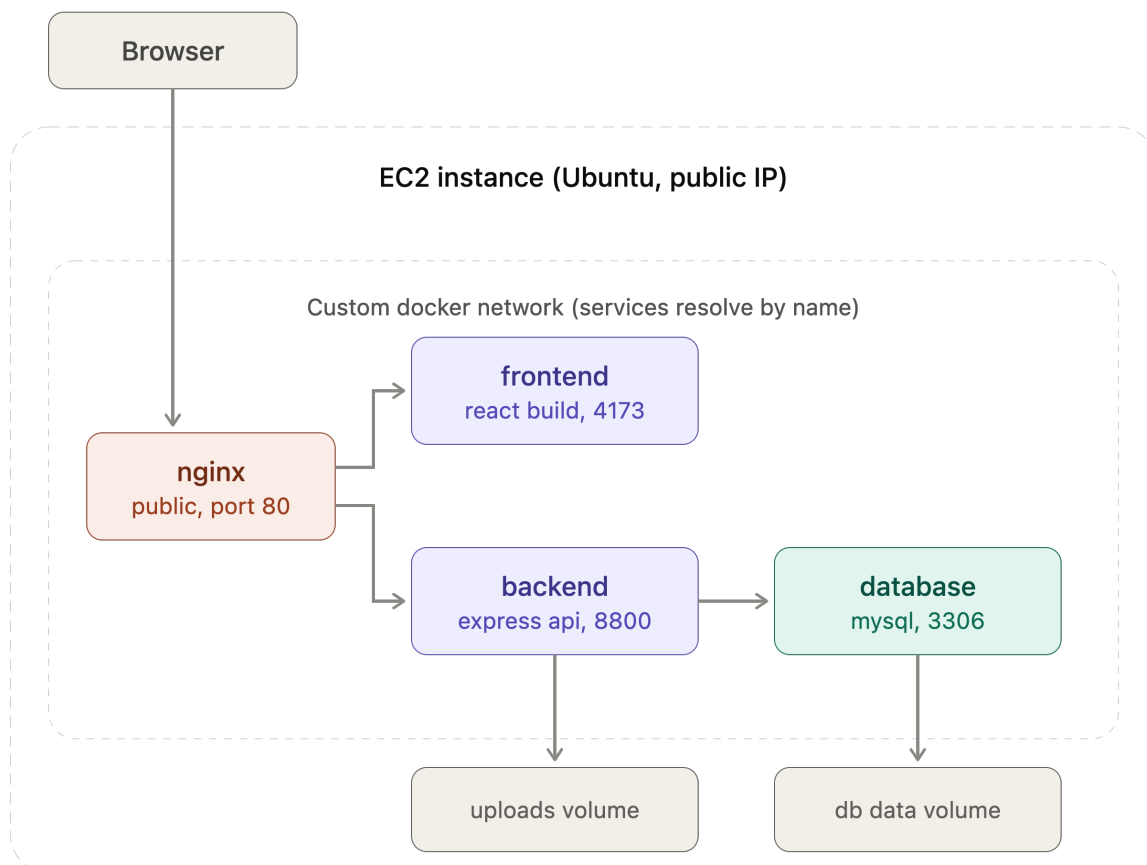
SocialPulse: Dockerized 3-Tier Deployment on AWS EC2

Deployment walkthrough and screenshots

Name	Snigdha Chaudhari
Date	30 August 2026
Original repo	https://github.com/xloyb/SocialPulse-React-Node.js-MySQL-Social-Media-App-Full-Stack-Social-Network-App-Open-Source
My fork	https://github.com/CrystallyRains/SocialPulse-Social-Network-App-Open-Source.git
Instance	T3.medium, Ubuntu, us-east-1
Stack	React frontend, Node/Express API, MySQL, Nginx reverse proxy

The app is a social network with a React frontend, an Express API and a MySQL database. It had no Dockerfile and no compose file in the original repo. I wrote a Dockerfile for the frontend and one for the API, put Nginx in front as a reverse proxy, and brought the whole thing up with a single compose file on an EC2 instance. Screenshots below follow the order I actually did it in.

System Design



1. Launching the EC2 instance

Free tier Ubuntu instance in my own AWS account. Key pair created at launch so I could SSH in. Connected over SSH:

```
Downloads % ssh -i "l111.pem" ubuntu@ec2-107-21-82-181.compute-1.amazonaws.com
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

* Documentation:  https://docs.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Sun Aug 30 18:06:26 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   96.4% of 6.61GB Processes:      120
Memory usage: 27%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.92.163

=> / is using 96.4% of 6.61GB

* Ubuntu Pro delivers the most comprehensive open source security and
  compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

121 updates can be applied immediately.
87 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

3 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Sun Aug 30 18:06:27 2026 from 223.185.39.128
ubuntu@ip-172-31-92-163:~$
```

2. Security group setup

Only the ports that need to be public are open. Port 80 for the app through Nginx, and SSH for me. The MySQL port is not open to the internet at all, the database is only reachable from the other containers on the internal Docker network.

The screenshot shows the AWS Management Console interface for a security group. The breadcrumb navigation at the top indicates the path: Security Groups > sg-07745368e8a4da611. The main header displays the security group name 'sg-07745368e8a4da611 - launch-wizard-10' and an 'Actions' dropdown menu.

The 'Details' section provides key information about the security group:

- Security group name:** launch-wizard-10
- Security group ID:** sg-07745368e8a4da611
- Description:** launch-wizard-10 created 2026-08-29T17:41:46.573Z
- VPC ID:** vpc-00909f8b416deb9d2
- Owner:** 252366102382
- Inbound rules count:** 5 Permission entries
- Outbound rules count:** 1 Permission entry

Below the details, there are tabs for 'Inbound rules', 'Outbound rules', 'Sharing', 'VPC associations', 'Related resources - new', and 'Tags'. The 'Inbound rules' tab is currently selected, showing a list of 5 rules.

The 'Inbound rules (5)' section includes a search bar and a table of rules:

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-01ffd1ddc64c2d7a5	IPv4	MYSQL/Aurora	TCP	3306
☐	-	sgr-0d8194398498b708d	IPv4	HTTP	TCP	80
☐	-	sgr-0f9c01236ca35807a	IPv4	Custom TCP	TCP	8800
☐	-	sgr-07e28cae48c63202e	IPv4	Custom TCP	TCP	4173
☐	-	sgr-0628fda2252de0bcb	IPv4	SSH	TCP	22

```

ubuntu@ip-172-31-92-163:~$ ls
SocialPulse-Social-Network-App-Open-Source
ubuntu@ip-172-31-92-163:~$ cd SocialPulse-Social-Network-App-Open-Source/
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$ ls
API LICENSE README.md ShowCase docker-compose.yml frontend mydevify_social.sql nginx
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$ ls frontend/
Dockerfile index.html package.json public tailwind.config.js
README.md package-lock.json postcss.config.js src vite.config.js
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$ ls API/
Dockerfile connect.js controllers index.js node_modules package-lock.json package.json routes
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$

```

```

Last login: Sun Aug 30 18:00:27 2020 from 228.100.07.120
ubuntu@ip-172-31-92-163:~$ docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1
ubuntu@ip-172-31-92-163:~$

```

3. Installing Docker and Compose

Installed Docker and the compose plugin, then added my user to the docker group so I did not have to sudo every command.

```
$ sudo apt update && sudo apt install docker.io -y
```

```
$ sudo usermod -aG docker $USER && newgrp docker
```

```
$ docker compose version
```

4. Pulling the project onto the instance

Cloned my fork, which already had both Dockerfiles, the compose file, the Nginx config and the SQL dump committed.

```
$ git clone https://github.com/CrystallyRains/SocialPulse-Social-Network-App-Open-Source.git
```

```

ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$ docker compose up -d
[+] Running 5/5
 ✓ Network socialpulse-social-network-app-open-source_socialpulse-net      Created           0.1s
 ✓ Container socialpulse-social-network-app-open-source-database-1         Healthy           6.0s
 ✓ Container socialpulse-social-network-app-open-source-frontend-1         Started           0.5s
 ✓ Container socialpulse-social-network-app-open-source-backend-1          Started           6.1s
 ✓ Container socialpulse-social-network-app-open-source-nginx-1            Started           6.3s
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$ docker compose ps
NAME                                IMAGE                                COMMAND
socialpulse-social-network-app-open-source-backend-1  socialpulse-social-network-app-open-source-backend  "docker-en
trypoint.s..." backend 12 seconds ago Up 6 seconds 8800/tcp
socialpulse-social-network-app-open-source-database-1  mysql:8.4                                             "docker-en
trypoint.s..." database 12 seconds ago Up 11 seconds (healthy) 3306/tcp, 33060/tcp
socialpulse-social-network-app-open-source-frontend-1  socialpulse-social-network-app-open-source-frontend  "docker-en
trypoint.s..." frontend 12 seconds ago Up 11 seconds 4173/tcp
socialpulse-social-network-app-open-source-nginx-1     nginx:alpine                                          "/docker-e
ntrypoint.s..." nginx 12 seconds ago Up 5 seconds 0.0.0.0:80->80/tcp, [::]:80->80/tcp
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$

```

5. Bringing the stack up

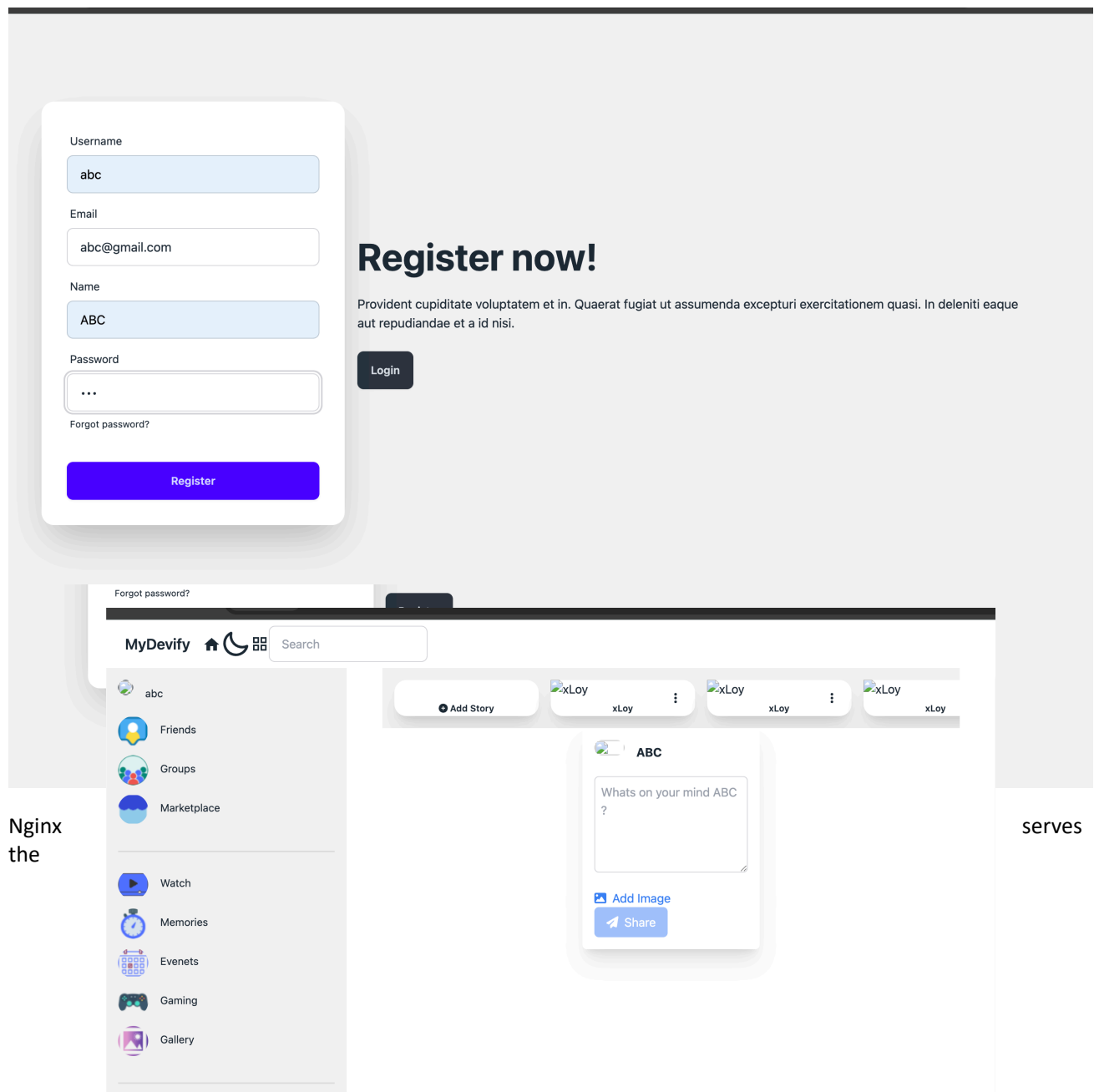
One command brings up all four services on a custom bridge network, so they reach each other by service name instead of hardcoded IPs.

```
$ docker compose up -d
```

All containers healthy and running:

```
$ docker compose ps
```

6. The app running through the public IP



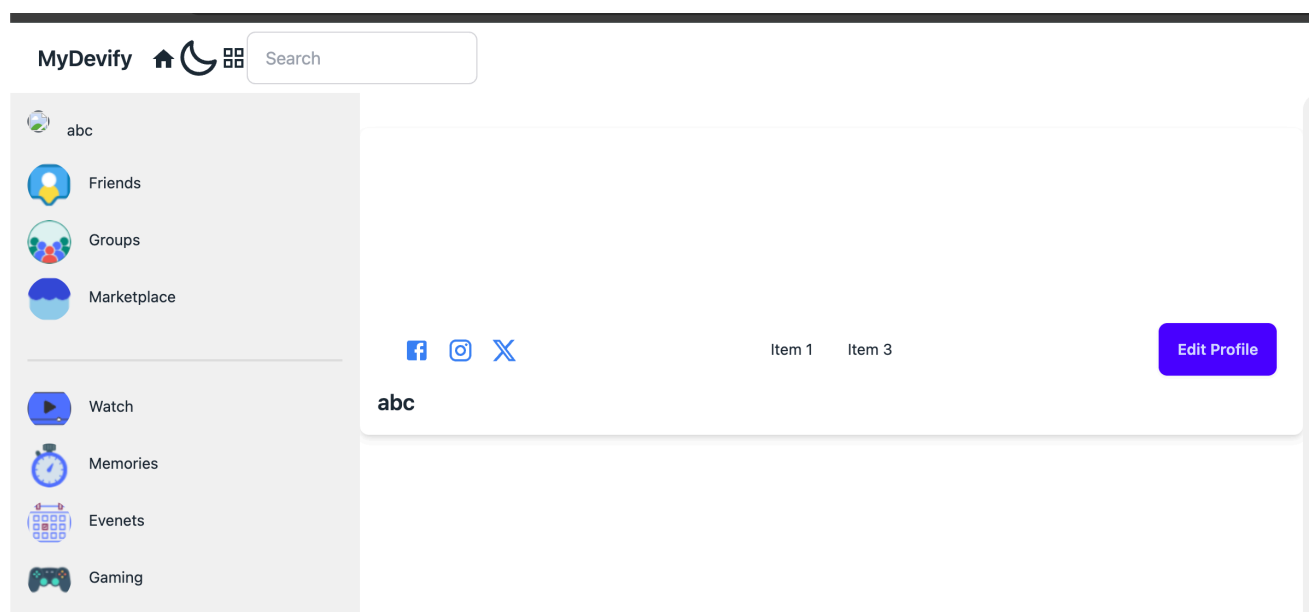
frontend on port 80 and proxies /api to the backend and /uploads to the uploaded images, so the browser only ever talks to one origin.

Registered a user and logged in, so the request actually reached the API and MySQL:

7. Persistence proof

The database uses a named volume, so the data lives outside the container. To prove it, I created data, destroyed the containers and brought them back up.

Data before:

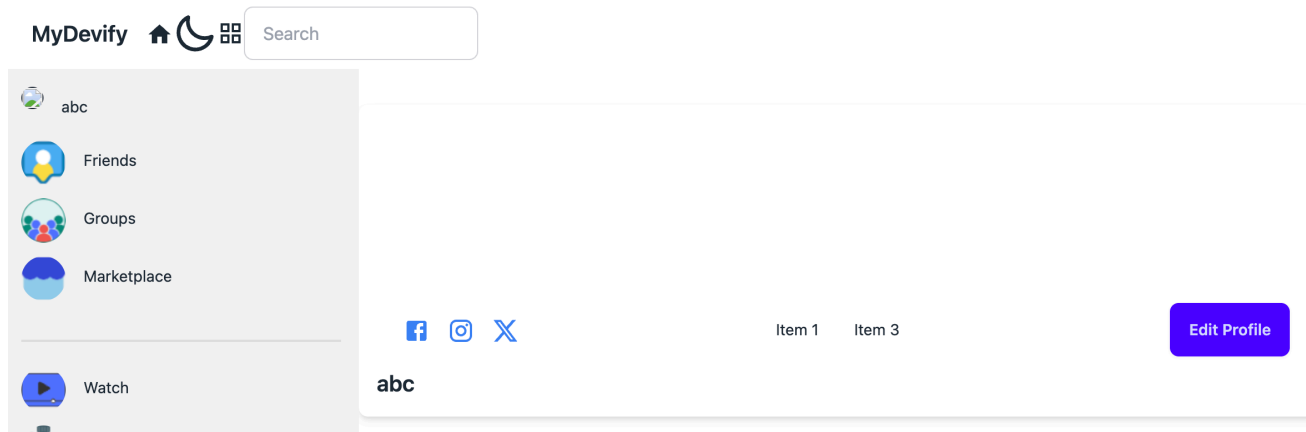


Containers removed and recreated:

```
$ docker compose down
```

```
$ docker compose up -d
```

```
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$ docker compose down
[+] Running 5/5
 ✓ Container socialpulse-social-network-app-open-source-nginx-1      Removed      10.2s
 ✓ Container socialpulse-social-network-app-open-source-frontend-1   Removed      0.7s
 ✓ Container socialpulse-social-network-app-open-source-backend-1    Removed      10.2s
 ✓ Container socialpulse-social-network-app-open-source-database-1    Removed      1.3s
 ✓ Network socialpulse-social-network-app-open-source_socialpulse-net Removed      0.1s
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$ docker compose up -d
[+] Running 5/5
 ✓ Network socialpulse-social-network-app-open-source_socialpulse-net Created      0.1s
 ✓ Container socialpulse-social-network-app-open-source-frontend-1   Started      0.6s
 ✓ Container socialpulse-social-network-app-open-source-database-1    Healthy      6.1s
 ✓ Container socialpulse-social-network-app-open-source-backend-1     Started      6.1s
 ✓ Container socialpulse-social-network-app-open-source-nginx-1       Started      6.3s
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$
```



Same data after the recreate:

Volume confirmed from the CLI as well:

```
$ docker volume ls
```

```
DOCKER: command not found
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$ docker volume ls
DRIVER      VOLUME NAME
local       7181e66b57dbc1ae9d6060f97abda9380b9c80eeb2ae1f3f955bbacc473812e8
local       fe983b4127f871d5e745757dc69cf28161cef447ab4432994eb3444b1d9c6fe4
local       socialpulse-social-network-app-open-source_mysql
local       socialpulse-social-network-app-open-source_uploads
ubuntu@ip-172-31-92-163:~/SocialPulse-Social-Network-App-Open-Source$
```

8. Things that broke along the way

Short notes on the problems I hit, since the fixes are part of the work:

- The backend crashed on startup because the repo still imported a Redis client that was never used. Removed it and the container came up clean.
- The API used the old mysql driver, which fails against MySQL 8 default auth. Swapped it for mysql2 and updated the connection file.
- The frontend had localhost URLs hardcoded for the API and for uploaded images, which works locally and breaks the moment you open it from another machine. Moved everything behind relative paths through Nginx.
- The database was not ready when the backend first tried to connect, so the connection had to wait for MySQL to actually accept connections rather than just for the container to exist.
- *Replace or trim these to match what you actually want to admit to. Keeping two or three real ones reads better than a clean run.*